OAP200V

Object-oriented analysis, design and programming

00 – Database access

Lecturer: Boban Vesin

Outline

- JDBC
- Connect to a MySQL database with Java
- Submit SQL statement to insert, update and delete data
- Handle SQL parameters with Prepared Statements

JDBC - Java Database Connectivity

- Allows Java applications to connect to a relational database
- Features
  - Provides portable access to various databases
    - Oracle, MySQL, SQL server, DB2, PostgreSQL
  - You can build your own custom SQL statements
- JDBC architecture
  - JDBC driver – provides connection to a database
    - Converts JDBC calls to for specific database
  - JDBC DriverManager
    - Helps to connect an application based on the database connection string

JDBC API

- JDBC API is defined in the following packages:
  - Java.sql and javax.sql
- Key classes:
  - java.sql.DriverManager
  - java.sql.Connection
  - java.sql.Statement
  - java.sql.ResultSet
  - java.sql.DataSource

Submitting SQL queries

Development process

1. Get a connection to database
2. Create a Statement object
3. Execute SQL query
4. Process the Result Set

1. Get a connection to database

- In order to connect to database
  - Need a connection string in form of JDBC URL
- Basic syntax:
  - jdbc:<driver protocol>:<driver connection details>
- Example (MySQL):
  - `jdbc:mysql://localhost:3306/demodb`

1. Get a connection to database

- Code snippet for connection to MySQL

```
import java.sql.*;

String dbUrl = "jdbc:mysql://localhost:3306/demo";
String user = "student";
String pass "student";

Connection myConn = DriverManager.getConnection(dbUrl, user , pass);
```

Step 2. Create a Statement object

- The Statement object is based on connection
  - It will be used later to execute SQL query

```
import java.sql.*;

String dbUrl = "jdbc:mysql://localhost:3306/demo";
String user = "student";
String pass "student";

Connection myConn = DriverManager.getConnection(dbUrl, user , pass);

Statement myStmt = myConn.createStatement();
```

Step 3. Execute SQL query

- Pass in your SQL query

```
import java.sql.*;

String dbUrl = "jdbc:mysql://localhost:3306/demo";
String user = "student";
String pass "student";

Connection myConn = DriverManager.getConnection(dbUrl, user , pass);

Statement myStmt = myConn.createStatement();

ResultSet myRs = = myStmt.executeQuery("select * from employees");
```

Step 4. Process Result Set

- Result Set is initially placed before first row
- Method: ResultSet.next();
  - Moves forward one row
  - Returns true if there are more rows to process
- Looping through a result set

```
ResultSet myRs = = myStmt.executeQuery("select * from employees");

while (myRs.next()) {
      // read data from each row
}
```

Step 4. Process Result Set

- Collection of methods for reading data
  - getXXX(columnName)
  - getXXX(columnIndex)

```
ResultSet myRs = = myStmt.executeQuery("select last_name, first_name from employees");

while (myRs.next()) {
      System.out.println(myRs.getString("last_name"));
      System.out.println(myRs.getString("first_name"));
}
```

Inserting data into the database

Development process

- Get connection
- Create a statement
- Execute SQL insert

Insert data

```
String dbUrl = "jdbc:mysql://localhost:3306/demo";
String user = "student";
String pass = "student";

myConn = DriverManager.getConnection(dbUrl, user, pass);
myStmt = myConn.createStatement();

int rowsAffected = myStmt.executeUpdate(
          "insert into employees " +
          "(last_name, first_name, email, department, salary) " +
          "values " +
          "('Wright', 'Eric', 'eric.wright@foo.com', 'HR', 33000.00)");
```

Updating data into the database

Development process

- Get connection
- Create a statement
- Execute SQL update statement

Update data

```
String dbUrl = "jdbc:mysql://localhost:3306/demo";
String user = "student";
String pass = "student";

myConn = DriverManager.getConnection(dbUrl, user, pass);
myStmt = myConn.createStatement();

int rowsAffected = myStmt.executeUpdate(
    "update employees " +
    "set email='john.doe@luv2code.com' " +
    "where last_name='Doe' and first_name='John'");
```

Deleting data from the database

Development process

- Get connection
- Create a statement
- Execute SQL delete statement

Delete data

```
String dbUrl = "jdbc:mysql://localhost:3306/demo";
String user = "student";
String pass = "student";

myConn = DriverManager.getConnection(dbUrl, user, pass);
myStmt = myConn.createStatement();

int rowsAffected = myStmt.executeUpdate(
        "delete from employees " +
        "where last_name='Doe' and first_name='John'");
```

Creating prepared statements

Prepared statements

- What are prepared statements
- Create statements
- Setting Parameter Values
- Executing prepared statement
- Reusing a prepared statement

Prepared statements

- A prepared statement is a precompiled SQL statement
- Prepared statement provide the following benefits
  - Makes it easier to setSQL parameter values
  - Prevent against SQL dependency injection attacks
  - May improve application performance
    - SQL statement is precompiled

Using prepared Statements

- Instead of hard coding your SQL values

```
select * from employees
where salary > 80000 and department = ‘Legal’
```

- Set parameters placeholders
  - Use question mark for placeholder: ?

```
select * from employees
where salary > ? and department = ?
```

Java coding

- Create the prepared statement

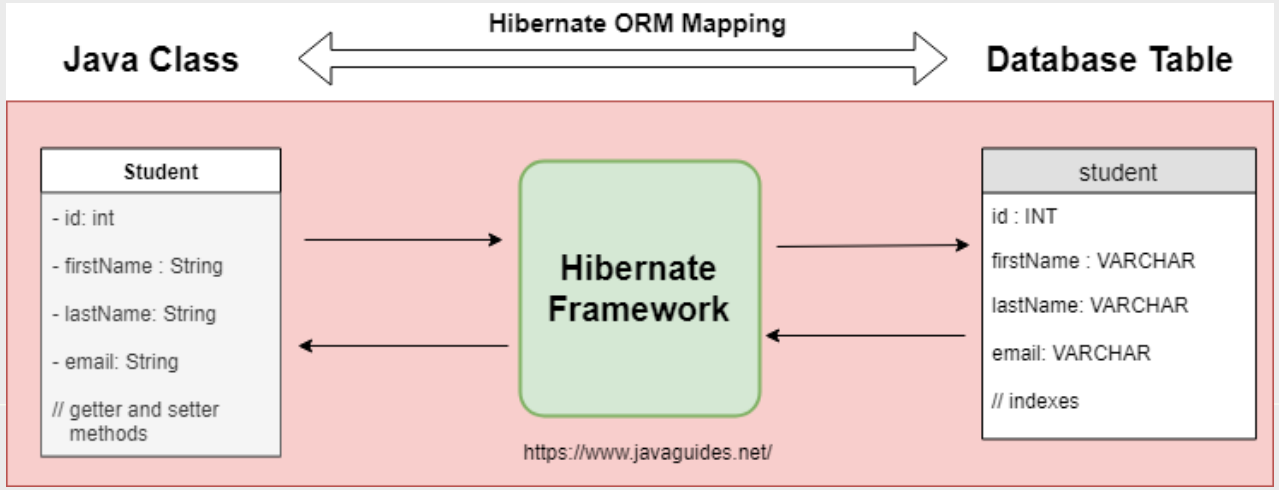
```
PreparedStatement myStmt =
     myConn.prepareStatement("select * from employees where salary > ? and department=?");

myStmt.setDouble(1, 80000);
myStmt.setString(2, "Legal");

ResultSet myRs = myStmt.executeQuery();
```

Alternative approaches to DB connectivity

- Java Persistence API (JPA)
  - provides a specification for persisting, reading, managing data from your Java object to relational tables in the database
- Hibernate
  - Object/Relational Mapping solution for Java environments
  - JPA implementation
  - ORM tool
- Spring data



Questions?